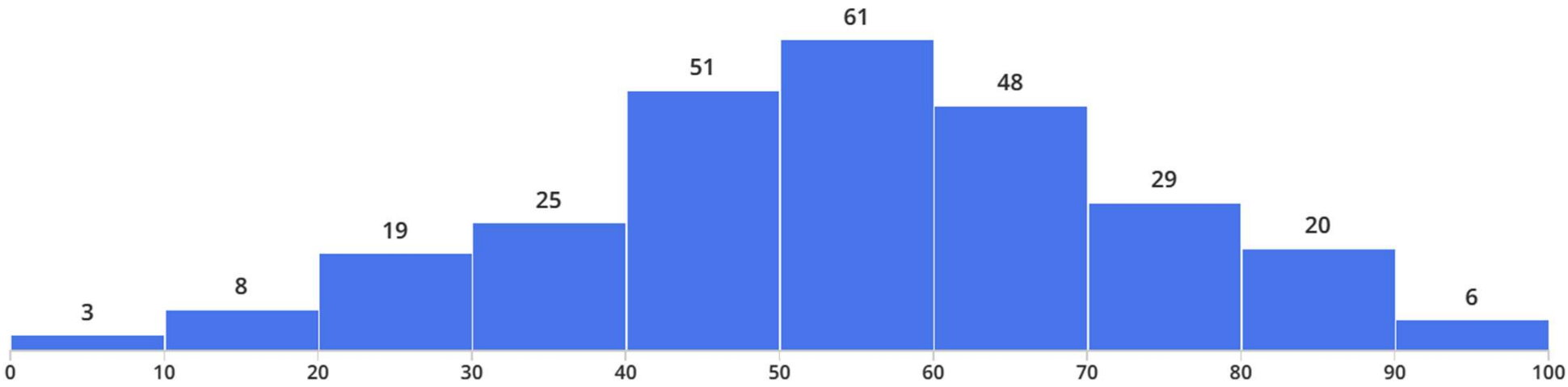
EECS 482
Introduction to operating systems

Post final exam

Peter Chen
University of Michigan

Final exam results



Minimum	Median	Maximum	Mean	Std Dev
0.0	54.0	98.0	53.82	19.02

1. RPC for a multi-threaded program (33 points)

You are given a multi-threaded program that runs on one computer and has the following function:

```
uint32_t myfunc(std::stack<uint32_t>);
```

`myfunc` is called by multiple threads concurrently. It is thread-safe and slow.

You may assume that the stack contains at most one million elements, and that `myfunc` uses no global variables. The beginning of the exam contains a list of functions for `std::stack`.

You would like to use RPC to allow `myfunc` to run on a different computer than the rest of the program. The computer that runs the rest of the program is called the *client*, and the computer that runs `myfunc` is called the *server*. You may assume the client and server have the same endianness.

- Key concepts
  - RPC
  - Network communication
  - Concurrent programming

```
unsigned int call_count=0;
mutex m;

uint32_t myfunc(std::stack<uint32_t> s) {
    m.lock();
    call_count++;
    m.unlock();

    // connect to server
    auto server_sock = socket(AF_INET, SOCK_STREAM, 0);
    connect(server_sock, server_sockaddr, sizeof(server_sockaddr));

    // send number of elements
    uint32_t stack_size = s.size();
    send(server_sock, &stack_size, sizeof(stack_size));

    // send stack elements
    while (!s.empty()) {
        auto e = s.top();
        send(server_sock, &e, sizeof(e));
        s.pop();
    }
```

```
    // get return value
    uint32_t status;
    recv(server_sock, &status, sizeof(status), MSG_WAITALL);

    close(server_sock);
    return(status);
}
```

```
void main() {
    while (auto s = accept(accept_socket, nullptr, nullptr)) {
        thread(handle, s).detach();
    }
}
```

```
void handle(socket s) {
    // receive number of elements
    uint32_t stack_size;
    recv(s, &stack_size, sizeof(stack_size), MSG_WAITALL);

    // receive the stack (top element first)
    std::stack<uint32_t> s1;
    for (unsigned int i=0; i<stack_size; i++) {
        // receive stack element
        uint32_t e;
        recv(server_sock, &e, sizeof(e), MSG_WAITALL);
        s1.push(e);
    }

    // reverse the stack
    std::stack<uint32_t> s2;
    while (!s1.empty()) {
        s2.push(s1.top());
        s1.pop();
    }

    // call myfunc
    uint32_t status = myfunc(s2);

    // send result
    send(s, &status, sizeof(status));
    close(s);
}
```

2. Reducing the overhead of allocating/initializing page tables (32 points)

The Project 3 pager managed a small portion of each process's address space (i.e., the arena), which led to small page tables. In this question, the pager manages the entire address space of each process. This increases the size of the page table, and in this question, you will consider how to reduce the overhead of allocating/initializing these larger page tables.

As with Project 3, the pager in this question deals only with application address spaces; you are not managing the kernel's address space.

Assume the system uses 40-bit addresses, a page size of 65536 (i.e., 2^{16}) bytes, and the same MMU algorithm and page table structure as Project 3 (shown below). **You may not change these properties.**

- Key concepts
  - Memory management
  - Page tables
  - Copy-on-write sharing

2.1 (4 points)

How many bytes are needed to store a page table for a process in this system? Your answer may be given as an arithmetic expression of numbers. Justify your answer.

- 2^{40} bytes in the address space
- Each PTE describes 1 page (2^{16} bytes)
- $2^{40}/2^{16} = 2^{24}$ PTEs in page table
- Each PTE is 4 bytes
- Page table is 2^{26} bytes

2.2 (6 points)

Assume you are starting with a pager that manages the entire address space (not just the arena) and handles child processes that are created by forking a parent process with a non-empty address space, but does not attempt to reduce overhead due to initializing page tables. Call this the *old pager*.

In this question, you will briefly (a couple sentences) describe your design for a *new pager* that reduces the overhead due to allocating/initializing page tables. As in Project 3, assume the most expensive operations are (from most expensive to least expensive) disk I/O, copying/initializing a page, and page faults. Remember that you may not change the properties of the system described above (40-bit addresses, page size of 65536, same MMU algorithm and page table structure as Project 3).

- Partial allocation of page table not allowed by MMU
- Multi-level page table not allowed by MMU
- Copy-on-write sharing

2.3.1

Describe how the new pager (revealed in the link to Question 2.2) carries out each step in the following sequence. Note any differences from the old pager. Label each part of your answer with the step number below.

1. Process PARENT has a non-empty address space and creates (i.e., forks) a process CHILD.
2. The kernel switches to process CHILD.
3. Process CHILD maps a swap-backed page.
4. Process CHILD reads the swap-backed page it just mapped.

Partially correct

1. Child shares page table with parent
2. PTBR points at child/parent page table
3. Copy page table and add new entry (r/!w)
4. No fault

Correct

1. Child shares page table with parent
2. PTBR points at child/parent page table
3. Pager leaves child/parent page table as shared, with swap-backed page as !r/!w
4. Page fault. Copy page table and add new entry as r/!w

2.3.2

Describe how the new pager (revealed in the link to Question 2.2) carries out each step in the following sequence. Note any differences from the old pager. Label each part of your answer with the step number below.

1. Process PARENT has a non-empty address space and creates (i.e., forks) a process CHILD.
2. The kernel switches to process CHILD.
3. Process CHILD maps a file-backed page that is non-resident. Assume the filename for this page is stored in a resident, referenced page.
4. Process CHILD reads the file-backed page it just mapped.

1. Child shares page table with parent
2. PTBR points at child/parent page table
3. No change. File-backed page is still !r/!w
4. Page fault. Copy page table; read file-backed page from disk; add new entry as r/w

2.3.3

Describe how the new pager (revealed in the link to Question 2.2) carries out each step in the following sequence. Note any differences from the old pager. Label each part of your answer with the step number below.

1. Process PARENT has a non-empty address space, which contains a file-backed page P that is non-resident. PARENT then creates (i.e., forks) a process CHILD.
2. The kernel switches to process CHILD.
3. Process CHILD reads file-backed page P.
4. Process CHILD is destroyed.

1. Child shares page table with parent
2. PTBR points at child/parent page table
3. Change file-backed page to r/w (applies to both child and parent)
4. Remove child's reference to shared page table

3. Atomic, durable disk writes (35 points)

For Project 4, we assumed that a single-block disk write was atomic and durable (in this question, *atomic* refers to an operation's behavior with respect to crashes). For this question, you will use a **non-atomic** disk, which does **not** provide atomicity for single-block disk writes. Instead, if a program issues a single-block disk write to block N with `NEW_DATA`, the non-atomic disk allows **three** possible outcomes, depending on when (or whether) the system crashes:

- **No change**: The state of block N is unchanged.
- **Success**: Block N contains `NEW_DATA`.
- **Corruption**: Block N is corrupt. In this case, subsequent reads of block N will return an error, until this block is written successfully.

- Key concepts
  - Persistent storage
  - Atomicity
  - Crash consistency

3.1 (5 points)

You have been asked to design a virtual disk that provides atomicity and durability, using only a non-atomic disk. Your design must support concurrent accesses to different virtual blocks, but **it need not support concurrent reads (or writes) to the same virtual block**. Your design may limit the virtual disk to fewer than 1024 blocks.

Describe how your design organizes data on the underlying non-atomic disk, and how many virtual blocks your design provides.

- Keep two copies of each block (supports 512 virtual blocks)

```
void virtual_readblock(unsigned int block, void* buf) {
    unsigned int primary = 2*block;
    unsigned int secondary = 2*block+1;

    // Try reading primary.  If this succeeds, we're done.
    if (!disk_readblock(primary, buf)) {
        // Primary is corrupt, so read secondary. This is guaranteed to
        // succeed.
        disk_readblock(secondary, buf);

        // Fix primary to speed up future reads.
        disk_writeblock(primary, buf);
    }
}
```

```
void virtual_writeblock(unsigned int block, void* buf) {
    unsigned int primary = 2*block;
    unsigned int secondary = 2*block+1;
    char tmp[FS_BLOCKSIZE];

    disk_writeblock(primary, buf);
    disk_writeblock(secondary, buf);
}
```

How could this break?

```
void virtual_writeblock(unsigned int block, void* buf) {
    unsigned int primary = 2*block;
    unsigned int secondary = 2*block+1;
    char tmp[FS_BLOCKSIZE];

    // Is primary good?
    if (disk_readblock(primary, tmp)) {
        // Primary is good.
        // Write secondary (which may be corrupt or old), then primary.
        disk_writeblock(secondary, buf); // could write tmp instead of buf
        disk_writeblock(primary, buf);

    } else {
        // Primary is corrupt, so secondary must be good.
        // Write primary (and there's no need to write secondary).
        disk_writeblock(primary, buf);
    }
}
```

Regrade requests

- First, make sure you understand the question and solution
- Explain why you believe your answer was graded incorrectly (should be clear grading error)
- We will re-grade entire question (your score may go up or down)

- One pending regrade request at a time. May submit another regrade request only if prior one is successful.
- Deadline for regrade requests: end of December 18